Dynamics and Stability of the Spring-Loaded Inverted Pendulum (SLIP) Model

1 Introduction and Relevance

The spring-loaded inverted pendulum (SLIP) is the reduced-order model behind most legged-robot dynamics [3, 5]: a point mass bouncing on a massless spring leg. Despite its simplicity, it captures the center-of-mass dynamics of running in both animals and machines remarkably well [1, 2]. That makes it a good subject for this course: it is simple enough to analyze fully with the tools we covered, but rich enough to produce genuinely interesting hybrid dynamics. It also builds directly on the elastic pendulum studied in class, since the stance phase of SLIP is an inverted spring pendulum.

What makes SLIP interesting from a mechanics standpoint is that it is a hybrid dynamical system: during stance the body behaves like a spring-loaded pendulum, while in flight it is a free projectile, with the two phases connected by touchdown and liftoff switching conditions. Rather than analyzing stability at a fixed equilibrium point, the more natural question for a hopping gait is whether the periodic motion itself is stable. The standard tool for that is the apex return map, a Poincaré map in which a steady hopping gait appears as a fixed point whose stability can be assessed numerically [1, 4].

Beyond being a convenient system, SLIP-type models are used directly in the design and control of running robots and in the biomechanical study of animal and human locomotion, which is my practical motivation for this project: a model simple enough to analyze by hand but predictive enough to inform how a real legged system should be built or controlled.

Relation to Prior Work

This project works within an established line of SLIP research, and each stage of the model draws on a different part of that literature. Blickhan [1] introduced the spring-mass template and showed that it reproduces the ground-reaction-force and center-of-mass patterns of human and animal running; the stance-phase Lagrangian derived here reproduces that same reduced-order model, but from Lagrange's equations in polar coordinates rather than Blickhan's Newtonian force balance. McMahon and Cheng [2] used the same template to relate leg stiffness to running speed empirically; the parameter sweep over stiffness k and touchdown angle θ_{TD} planned for this project asks the same question, but numerically, from the model directly, rather than from experimental data. Full and Koditschek's templates-and-anchors framework [3] is the justification for treating SLIP as a legitimate object of study on its own, rather than only as a shortcut on the way to a fully articulated, multi-link leg model: a template is expected to capture the essential dynamics of the anchored system while discarding detail that does not matter for stability. Schwind and Koditschek [4] derived an approximate, closed-form stance map for SLIP using a perturbative solution; this project instead integrates the stance-phase equations numerically with an event-based solver, trading their analytical insight for an exact (if less interpretable) stance map, consistent with the computational tools emphasized in this course. Saranli, Buehler, and Koditschek [5] built RHex, a hexapod whose individual legs behave like a SLIP template during stance, which is the practical grounding for why this abstraction is worth studying: it is not just a toy system, it is the model actually used to understand a working robot. Finally, Geyer, Seyfarth, and Blickhan [6] extended a SLIP-type compliant-leg model to walking, not just running, by allowing double support; that extension is a natural direction for future work beyond the single-support hopping model considered here.

2 Modeling of Dynamics

The equations of motion for both phases of the SLIP model are derived using **Lagrange's equations of motion**, with generalized coordinates chosen separately for each phase: Cartesian coordinates (x, y)

for the flight phase, and polar coordinates (r, θ) about the planted foot for the stance phase. The two phases are connected by touchdown and liftoff switching (transversality) conditions, so the overall model is a hybrid dynamical system integrated with event-based numerical methods. Figure 1 shows the geometry of both phases.

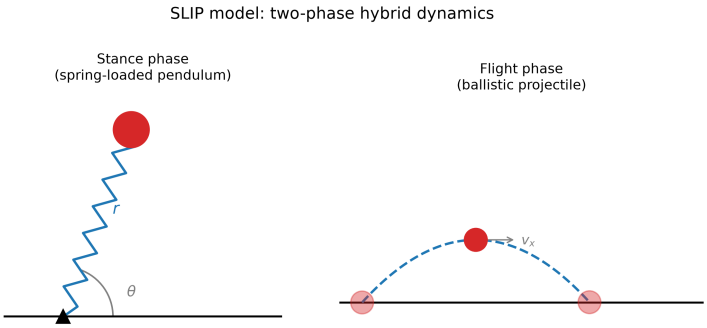

Figure 1: SLIP model geometry: stance (spring pendulum, r, θ) and flight (projectile).

2.1 Flight-Phase Equations of Motion

With basis $\hat{n}_1, \hat{n}_2$, $\vec{r}_m = x\hat{n}_1 + y\hat{n}_2$:

$$T = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2), \qquad V = mgy, \qquad L = T - V = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) - mgy$$

Lagrange's equations give

$$\ddot{x} = 0, \qquad \ddot{y} = -g$$

Starting at the apex ($\dot{y} = 0$), with $x(0) = x_0$, $\dot{x}(0) = v_{x0}$:

$$x(t) = v_{x0}t + x_0, \qquad y(t) = y_0 - \frac{g}{2}t^2$$

2.2 Touchdown: Detection and Transition Map

θ_{TD} is held fixed in flight, so the foot trails the body at

$$\vec{r}_{foot}(t) = (x(t) - r_0 \cos\theta_{TD})\hat{n}_1 + (y(t) - r_0 \sin\theta_{TD})\hat{n}_2$$

Touchdown is $y_{foot} = 0$, i.e. $y_{TD} = r_0 \sin\theta_{TD}$. Solving $y(t) = y_0 - \frac{g}{2}t^2$:

$$t_{TD} = \sqrt{\frac{2(y_0 - r_0 \sin\theta_{TD})}{g}}, \qquad \dot{y}_{TD} = -\sqrt{2g(y_0 - r_0 \sin\theta_{TD})}, \qquad x_{TD} = v_{x0}t_{TD}, \qquad \dot{x}_{TD} = v_{x0}$$

Stance uses polar coordinates about the planted foot, so this Cartesian state must be converted. With foot at $\vec{r}_F = (x_{TD} + r_0 \cos\theta_{TD})\hat{n}_1$ and leg vector $\vec{\rho} = \vec{r}_m - \vec{r}_F = -r_0 \cos\theta_{TD}\,\hat{n}_1 + y_{TD}\hat{n}_2$:

$$r = |\vec{\rho}| = \sqrt{r_0^2 \cos^2\theta_{TD} + y_{TD}^2} = r_0, \qquad \theta = \operatorname{atan2}(y_{TD},\, -r_0 \cos\theta_{TD})$$

Projecting $\vec{v} = \dot{x}_{TD}\hat{n}_1 + \dot{y}_{TD}\hat{n}_2$ onto $\hat{e}_r = \cos\theta\,\hat{n}_1 + \sin\theta\,\hat{n}_2$, $\hat{e}_\theta = -\sin\theta\,\hat{n}_1 + \cos\theta\,\hat{n}_2$:

$$\dot{r} = \vec{v} \cdot \hat{e}_r = \dot{x}_{TD}\cos\theta + \dot{y}_{TD}\sin\theta, \qquad \dot{\theta} = \frac{\vec{v} \cdot \hat{e}_\theta}{r_0} = \frac{-\dot{x}_{TD}\sin\theta + \dot{y}_{TD}\cos\theta}{r_0}$$

implemented as `touchdownState()`.

2.3 Stance-Phase Equations of Motion

In stance the leg pivots about the fixed foot: $\vec{r}_m = r\hat{e}_r$, $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$:

$$T = \frac{1}{2}m\dot{r}^2 + \frac{1}{2}mr^2\dot{\theta}^2, \qquad V = mgr\sin\theta + \frac{1}{2}k(r - r_0)^2, \qquad L = T - V$$

Lagrange's equations:

$$m\ddot{r} - mr\dot{\theta}^2 + mg\sin\theta + k(r - r_0) = 0, \qquad mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} + mgr\cos\theta = 0$$

solved for the highest derivatives:

$$\ddot{r} = r\dot{\theta}^2 - g\sin\theta - \frac{k}{m}(r - r_0), \qquad \ddot{\theta} = -\frac{2}{r}\dot{r}\dot{\theta} - \frac{g}{r}\cos\theta$$

implemented as `standingState()`.

2.4 Conserved Energy in Stance

$\vec{r}_m = r\hat{e}_r$ is scleronomic and T is homogeneous quadratic in $(\dot{r}, \dot{\theta})$, so $H = T + V$; with no explicit t-dependence, energy is conserved through stance:

$$E = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr\sin\theta + \frac{1}{2}k(r - r_0)^2 = \text{const.}$$

Neither coordinate is cyclic, so both equations must be integrated together; this conserved quantity is used below as a check on the numerical integrator.

2.5 Liftoff Map and the Apex Return Map

Liftoff occurs at $r = r_0,\ \dot{r} > 0$. Chaining flight → touchdown → stance → liftoff → flight once defines the apex-to-apex map

$$P : (y_n, v_{x,n}) \mapsto (y_{n+1}, v_{x,n+1}),$$

whose fixed points are steady periodic hopping gaits and whose stability is governed by the linearization of P there. Section 2.7 reduces this to a single scalar, so that linearization is one derivative rather than a Jacobian.

Liftoff position converts to Cartesian from $\vec{r}_m = r\hat{e}_r$ at $r = r_0$, $\theta = \theta_{LO}$:

$$x_{LO} = r_0\cos\theta_{LO}, \qquad y_{LO} = r_0\sin\theta_{LO}$$

Velocity converts by substituting $\hat{e}_r = \cos\theta\,\hat{n}_1 + \sin\theta\,\hat{n}_2$, $\hat{e}_\theta = -\sin\theta\,\hat{n}_1 + \cos\theta\,\hat{n}_2$ into $\vec{v}_m = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta$ and evaluating at liftoff:

$$\dot{x}_{LO} = \dot{r}_{LO}\cos\theta_{LO} - r_0\dot{\theta}_{LO}\sin\theta_{LO}, \qquad \dot{y}_{LO} = \dot{r}_{LO}\sin\theta_{LO} + r_0\dot{\theta}_{LO}\cos\theta_{LO}$$

Once airborne, apex is defined in the ground frame by $\dot{y} = 0$. With $\ddot{y} = -g$ and the clock started at liftoff, $\dot{y}(t) = \dot{y}_{LO} - gt$, so

$$t_{apex} = \frac{\dot{y}_{LO}}{g}$$

Propagating the flight solution $x(t) = \dot{x}_{LO}t + x_{LO}$, $y(t) = y_{LO} + \dot{y}_{LO}t - \frac{g}{2}t^2$ to $t = t_{apex}$:

$$x_{n+1} = x_{LO} + \dot{x}_{LO}\, t_{apex}, \qquad y_{n+1} = y_{LO} + \dot{y}_{LO}\, t_{apex} - \frac{g}{2}t_{apex}^2, \qquad v_{x,n+1} = \dot{x}_{LO}, \qquad \dot{y}_{n+1} = 0$$

Both are exact: $v_{x,n+1} = \dot{x}_{LO}$ since horizontal motion is unforced in flight ($\ddot{x} = 0$), and $\dot{y}_{n+1} = 0$ by construction of t_{apex}. x_{LO} is measured relative to the stance foot just released rather than a fixed ground origin. THis is consistent, since x never enters the dynamics of either phase (Section 2.7).

2.6 Conserved Energy at Apex and Across a Full Hop Cycle

By the same argument as Section 2.4, the flight Lagrangian is scleronomic with T homogeneous quadratic in $(\dot{x}, \dot{y})$, so $H = T + V$ is conserved through flight:

$$E = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2) + mgy = \text{const.}$$

with no spring term, since the leg carries no elastic energy while airborne. This differs from the stance energy $E_{stance} = \frac{1}{2}m(\dot{r}^2 + r^2\dot{\theta}^2) + mgr\sin\theta + \frac{1}{2}k(r - r_0)^2$ only by that spring term, which vanishes at $r = r_0$ – exactly the touchdown and liftoff condition. The two functionals therefore agree at every switching event, so total mechanical energy is continuous across touchdown and liftoff, and conserved over the full cycle apex → touchdown → stance → liftoff → apex. Section 4.2 validates this numerically.

2.7 Dimension Reduction of the Apex State

At apex, $\dot{y} = 0$ by definition. Substituting into the apex energy of Section 2.6 and solving for $\dot{x}$:

$$E - mgy = \frac{1}{2}m\dot{x}^2 \quad \Longrightarrow \quad \dot{x}^2 = \frac{2(E - mgy)}{m} = \frac{2E}{m} - 2gy$$

$$\dot{x} = \sqrt{\frac{2E}{m} - 2gy} \qquad (\dot{x} > 0 \text{ for forward hopping})$$

So the apex state $(x, y, \dot{x}, \dot{y})$ is not four independent numbers: $\dot{y} = 0$ always, x is ignorable (it enters no potential or force in either phase), and $\dot{x}$ is fixed by y once E is fixed. At fixed E, the apex state collapses to the scalar y, and the hop-to-hop dynamics reduce to the one-dimensional map

$$y_{n+1} = P(y_n)$$

where P is the composition: reconstruct $\dot{x}_n$ from y_n (above) → flight to touchdown (Section 2.2) → stance to liftoff (Section 2.3) → flight to the next apex (Section 2.5). Section 4.2 confirms this reduction numerically: $\dot{x}$ evaluated at the simulated output height y_{n+1} matches the simulator's own $\dot{x}_{n+1}$ to machine precision.

2.8 The Apex Return Map: Fixed Point and Stability Criterion

Using the reduction of Section 2.7, define the apex return map P at fixed leg stiffness k, fixed touchdown angle θ_{TD}, and fixed total energy E:

$$P : y_n \mapsto y_{n+1}$$

E characterizes how energetic a gait is; k and θ_{TD} are the model's only free parameters. There is no controller, so a periodic gait at a given E is whatever P produces for a chosen (k, θ_{TD}).

A periodic gait is a fixed point $y^* = P(y^*)$, equivalently a root of

$$f(y) = P(y) - y$$

Local stability is governed by the derivative at the fixed point, estimated by a central finite difference:

$$P'(y^*) \approx \frac{P(y^* + dy) - P(y^* - dy)}{2\,dy}$$

$$|P'(y^*)| < 1 : \text{ stable}, \qquad |P'(y^*)| > 1 : \text{ unstable}$$

$P'(y^*) < 0$ indicates oscillatory divergence/convergence (perturbations alternate sides of y^*); $P'(y^*) > 0$ indicates monotonic divergence/convergence. Section 4.3 reports the root-finding methodology and the result for the validated parameter set of Section 4.1.

3 Discussion of Assumptions

Several modeling choices were made to keep the SLIP model tractable with the tools of this course, each with a corresponding limitation:

- **Planar motion.** The model is restricted to the sagittal plane. Real running involves some lateral and rotational motion of the trunk and legs, which this model cannot capture.
- **Point mass, massless leg.** All body mass is lumped at a single point, and the leg itself carries no mass or rotational inertia. This isolates the center-of-mass dynamics that SLIP is meant to describe, but discards any effect of leg-swing inertia or trunk pitch, which the templates-and-anchors view [3] explicitly treats as detail that a template is allowed to omit.
- **Linear (Hookean) leg spring.** The stance leg is modeled as a single ideal linear spring, $F = -k(r - r_0)$, with constant stiffness k. Biological and robotic legs have leg stiffness that is nonlinear and can be actively modulated by the neuromuscular system or a controller; treating k as a fixed parameter to be swept is a simplification.
- **Fixed touchdown angle held through flight.** θ_{TD} is assumed constant for the entire flight phase, i.e. there is no swing-leg retraction or angle adjustment in response to a disturbance before the foot lands. This is the standard SLIP assumption, but it means the model's only "control input" between hops is the touchdown angle chosen before each flight, not a continuous correction during flight.
- **Instantaneous, rigid, non-slipping foot placement.** Touchdown and liftoff are treated as instantaneous events with no impact losses, and the foot is assumed to stick to the ground (no slipping, no ground compliance) for the duration of stance at a fixed height $y = 0$. Real contact involves finite compliance and a friction cone that could, in principle, be violated at high touchdown angles or high speeds.

- **No stance-phase dissipation, no aerodynamic drag.** The stance-phase Lagrangian has no damping term, so mechanical energy is exactly conserved within a single stance phase (verified numerically in Section 4); flight is likewise treated as drag-free projectile motion. This was a deliberate choice: it isolates the elastic dynamics of interest and gives a clean, near-machine-precision check on the numerical integrator, at the cost of neglecting the fact that any real leg or running surface dissipates some energy each stride.
- **Equal rest length at touchdown and liftoff.** The leg is assumed to return to the same natural length r_0 between hops, with no active leg extension/retraction actuation changing that rest length from one stance phase to the next.

4 Computational Results

4.1 Single-Stance Validation

The model is implemented in Python. `main.py` maps an apex state to the stance-phase initial condition via `touchdownState()`, then integrates `standingState()` with `solve_ivp` (`rtol` $= 10^{-10}$, `atol` $= 10^{-12}$) to the liftoff event $r = r_0,\ \dot{r} > 0$, checking the conserved energy from Section 2.4 at every step. Using the parameters below, this validates the stance-phase model end to end:

m	r_0	k	g	θ_{TD}	y_0	v_{x0}
80 kg	1 m	20000 N/m	9.81 m/s^2	68°	1.2 m	3.0 m/s

	r [m]	θ [deg]	$\dot{r}$ [m/s]	$\dot{\theta}$ [rad/s]
Touchdown	1.0000	112.00	−3.2689	−1.9149
Liftoff	1.0000	83.48	3.1937	−1.6886

Stance lasts 0.2094 s; the leg compresses to 0.7740 m (22.6% of r_0); relative energy drift is 6.46×10^{-13}, i.e. machine precision, confirming that the integrator and the equations of motion in Section 2.3 are consistent. Figure 2 plots leg length, angle, CoM trajectory, and energy drift for this run.

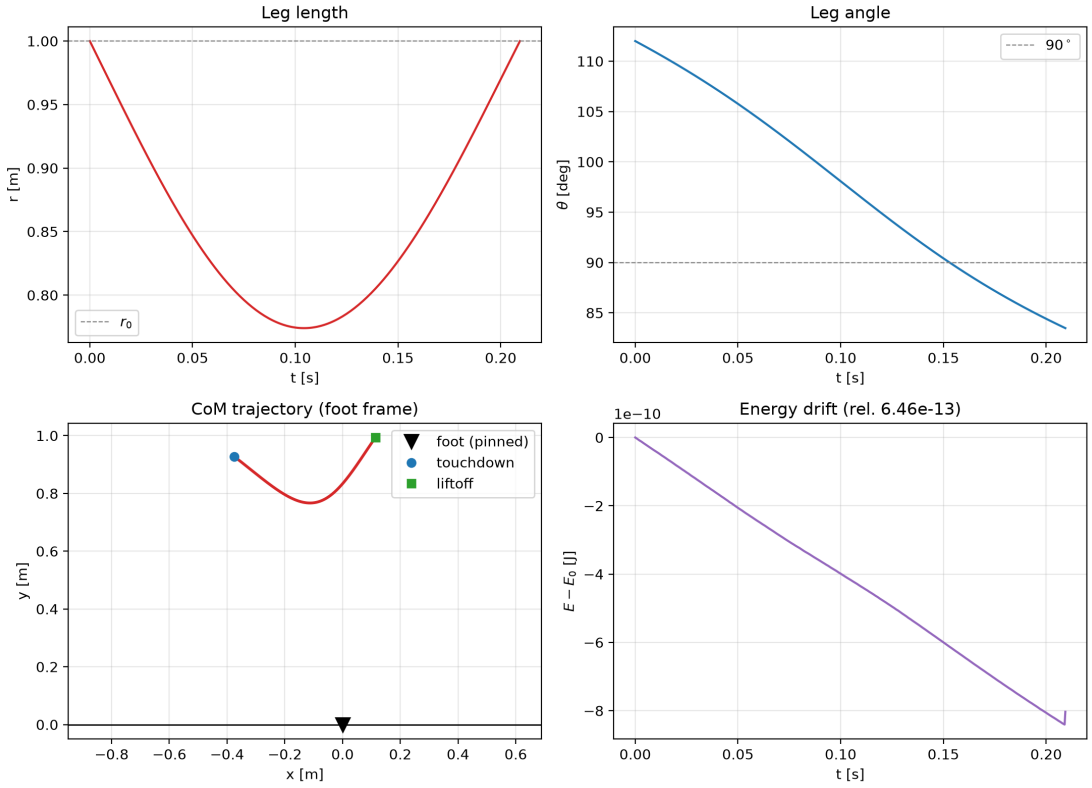

Figure 2: Stance simulation: $r(t)$, $\theta(t)$ (top), CoM trajectory (bottom left), energy drift (bottom right).

4.2 Full-Cycle Energy Conservation and Dimension-Reduction Validation

`proveX_dotIsRedundant()` validates the full hop cycle: starting from the apex state of Section 4.1, it computes the apex energy E_1 (`energyHop()`), chains one hop – flight → touchdown → stance → liftoff → flight (`hop()`) – and recomputes the apex energy E_2. Relative drift over the full cycle,

$$\frac{|E_2 - E_1|}{|E_1|} = 6.17 \times 10^{-13},$$

is consistent with the stance-only drift of Section 4.1 (6.46×10^{-13}), confirming total mechanical energy is conserved across the full switching cycle, not just within stance.

The same run checks the dimension reduction of Section 2.7: with E_1 fixed, $\dot{x} = \sqrt{2E_1/m - 2gy}$ evaluated at the simulated output height $y_{n+1} = 1.4465$ m predicts $\dot{x}_{pred} = 2.0403$ m/s, matching the simulator's own $v_{x,n+1} = 2.0403$ m/s to

$$\frac{|v_{x,n+1} - \dot{x}_{pred}|}{|\dot{x}_{pred}|} = 2.41 \times 10^{-12}.$$

4.3 Apex Return Map: Root-Finding Methodology and Result

`findApexMapFixedPoint()` locates the root of $f(y) = P(y) - y$ (Section 2.8) over the domain of y physically valid for a given (k, θ_{TD}, E): bounded below by the minimum apex height consistent with touchdown at angle θ_{TD},

$$y_{min} = r_0 \sin\theta_{TD},$$

and above by the height at which the closed-form $\dot{x}$ of Section 2.7 would require a negative value under the square root,

$$y_{max} = \frac{E}{mg}.$$

A small inward padding ($\epsilon = 10^{-3}$ m) is added to y_{min} to avoid floating-point roundoff pushing the touchdown free-fall relation's square root slightly negative at the exact boundary.

$f(y)$ is sampled coarsely (30 points) across $[y_{min}, y_{max}]$; once a bracket with a sign change is found, Brent's method refines it to a root y^*, and $P'(y^*)$ is estimated by the central difference of Section 2.8 with $dy = 10^{-3}$ m.

For the validated parameter set of Section 4.1 ($m = 80$ kg, $r_0 = 1$ m, $k = 20000$ N/m, $\theta_{TD} = 68°$, E fixed by $y_0 = 1.2$ m, $v_{x0} = 3.0$ m/s):

$$y^* \approx 1.5096 \text{ m}, \qquad P'(y^*) \approx -2.76$$

$|P'(y^*)| > 1$: this fixed point is unstable, and the negative sign indicates oscillatory divergence – a perturbed apex height overshoots on alternating sides of y^*, growing each hop. Steady hopping at $\theta_{TD} = 68°$, $k = 20000$ N/m is therefore not stable.

4.4 Grid Sweep Methodology and Parameter Sweep

`gridSearchFixedPoints()` wraps the pipeline of Section 4.3 into a function of (k, θ_{TD}) returning $(y^*, P'(y^*))$, or null if no sign change in $f(y)$ is found – i.e. no periodic gait exists there at this E. Each cell uses its own copy of the parameter dictionary, so sweeping k never mutates state shared across cells. `stableBandWidth()` additionally reports the longest run of contiguous stable cells along each grid direction, which is used below to tell a genuinely narrow stability margin from a coarse-grid artifact.

An initial coarse 40×40 diagnostic grid over $k \in [5000, 40000]$ N/m, $\theta_{TD} \in [55°, 80°]$ turned up only a one-cell-wide staircase of stable cells (band width 2 cells along θ_{TD}, 3 along k) threading through an otherwise unstable field – too thin to trust at that resolution. `localRefinementSweep()` zoomed a 20×20 window ($\pm 2.5° \times \pm 2500$ N/m, i.e. roughly $4\times$ finer) onto a representative stable cell near $k \approx 20256$ N/m, $\theta_{TD} \approx 71.0°$; there the stable band *thickened* (3 cells along θ_{TD}, 5 along k), meaning the coarse staircase was a grid-resolution artifact rather than the true boundary shape, so the full sweep needed that finer spacing before its shape could be trusted.

The final grid matches that resolution across the whole domain: a 133×95 sweep ($k \in [5000, 40000]$ N/m in steps of ≈ 265 N/m, $\theta_{TD} \in [55°, 80°]$ in steps of $\approx 0.27°$; 12,635 cells, of which only 5 found no fixed point), classified stable/unstable/no-fixed-point in Figure 3, with the friction-cone floor from Section 4.3 ($\theta_{TD} \geq 59°$, $\mu = 0.6$) overlaid. At this resolution the band widens slightly (3 cells along θ_{TD}, 8 along k) but is still only a few cells across. A second local refinement at the same fine spacing (Figure 4), now centered on the final grid's own representative stable cell ($k \approx 20114$ N/m, $\theta_{TD} \approx 70.96°$, where $y^* \approx 0.9496$ m and $P'(y^*) \approx 0.348$), gives a band width of 3 cells along θ_{TD} and 5 along k – the same order as the full grid, not larger, so the earlier thickening has stopped: the self-stable region is a real, if narrow, sliver of (k, θ_{TD}) space at this E, not an artifact of grid spacing.

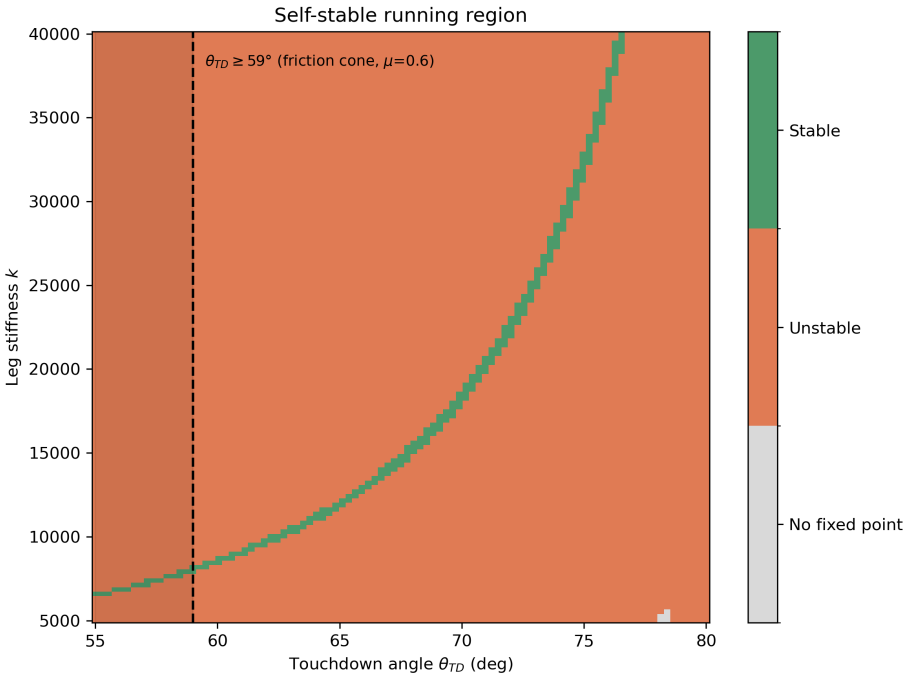

Figure 3: Self-stable running region over the full 133×95 (k, θ_{TD}) grid at fixed E (Section 4.1 parameters). Green: stable fixed point ($|P'(y^*)| < 1$); orange: unstable; gray: no fixed point found. Dashed line/shading: the friction-cone floor $\theta_{TD} \geq 59^\circ$ ($\mu = 0.6$).

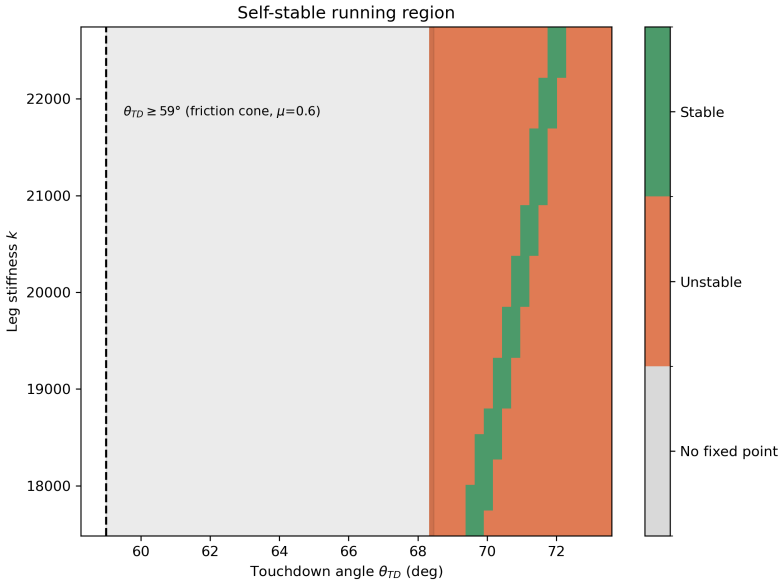

Figure 4: Local refinement (20×20, $\pm 2.5^\circ \times \pm 2500$ N/m) around the stable band near $k \approx 20114$ N/m, $\theta_{TD} \approx 70.96^\circ$, at the same fine spacing used to validate Figure 3's resolution. The band's width here matches the full grid, confirming it is not still thickening.

4.5 Apex-Map Iteration: Cobweb Diagrams for the Stable and Unstable Cases

Section 4.3 and Section 4.4 each report a fixed point and a slope, but neither shows what convergence or divergence of the apex map actually looks like hop to hop. `plotApexMapIteration()` answers that

directly by iterating $y_{n+1} = P(y_n)$ from a perturbed initial height and plotting both the cobweb diagram (the map $P(y)$, the diagonal $y_{n+1} = y_n$, and the resulting staircase) and y_n vs. hop index n, for two cases at the same fixed energy E: the stable representative cell of Section 4.4 ($k \approx 20114$ N/m, $\theta_{TD} \approx 70.96°$, $y^* \approx 0.9496$ m, $P'(y^*) \approx 0.35$) and the unstable validated point of Section 4.3 ($k = 20000$ N/m, $\theta_{TD} = 68°$, $y^* \approx 1.5096$ m, $P'(y^*) \approx -2.76$).

Figure 5 confirms the qualitative prediction of Section 2.8 from the sign and magnitude of $P'(y^*)$ alone: perturbing the stable case by +0.05 m produces a monotonic staircase that steps down the same side of the diagonal every hop, converging smoothly onto y^*. That this monotonic decay only shows up for a *small* perturbation is itself informative: the sweep only certifies *local* stability, and a larger perturbation of the same stable fixed point instead wanders through the map's global nonlinearity (the hump shape of $P(y)$ visible in the figure) before it happens to land back near y^*, if it does at all. Perturbing the unstable case by only +0.02 m, by contrast, produces a staircase that immediately crosses to the other side of the diagonal every hop and spirals outward, consistent with the oscillatory divergence already reported qualitatively in Section 4.3.

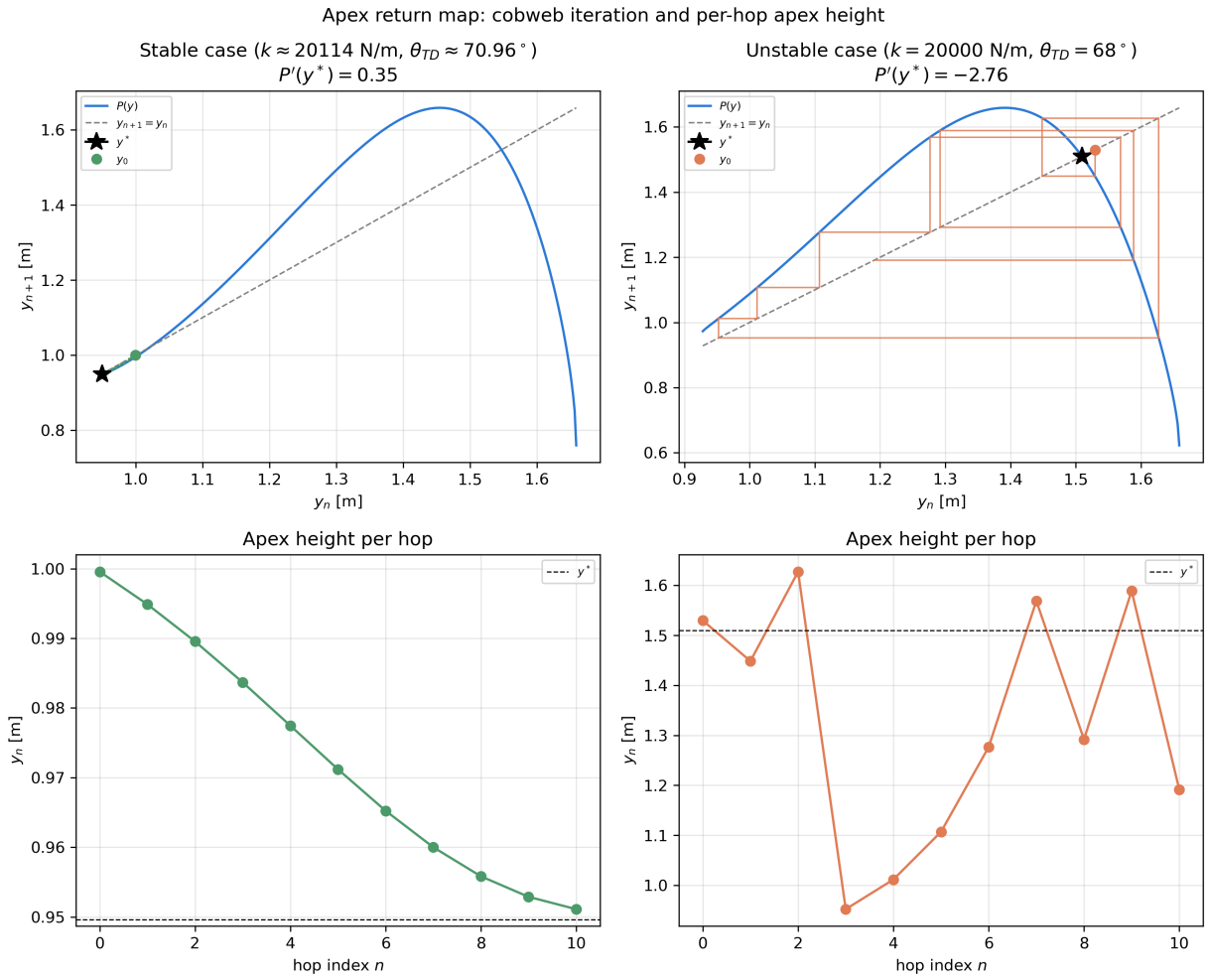

Figure 5: Apex-map cobweb iteration (top) and apex height per hop (bottom) for the stable cell of Section 4.4 (left, green) and the unstable point of Section 4.3 (right, orange). The stable case converges monotonically for a small perturbation; the unstable case diverges, alternating sides of y^* each hop.

4.6 Bifurcation Diagrams Along Each Sweep Direction

Section 4.4's grid already classifies every (k, θ_{TD}) cell as stable/unstable/no-fixed-point, but a single 2-D classification map does not show *how* the fixed point loses stability as a parameter is varied continuously. `plotBifurcationDiagrams()` slices two 1-D bifurcation diagrams directly out of that same grid – no new integration, just a row and a column read back out – both passing through the representative stable cell: y^* and $P'(y^*)$ vs. θ_{TD} at fixed $k \approx 20114$ N/m, and vs. k at fixed $\theta_{TD} \approx 70.96°$ (Figure 6).

Both slices show the same structure: $y^*(\theta_{TD})$ and $y^*(k)$ vary smoothly and almost linearly, with no branching or discontinuity in the fixed point itself, so this is not a classical saddle-node or pitchfork bifurcation in y^*. Instead the loss of stability lives entirely in $P'(y^*)$: it crosses $+1$ sharply at both edges of the narrow stable window (the thin green band already visible in Figure 3), confirming that the self-stable region identified by the grid sweep is bounded by genuine $|P'(y^*)| = 1$ crossings at both the θ_{TD} and k edges of the band, not by a grid-resolution artifact.

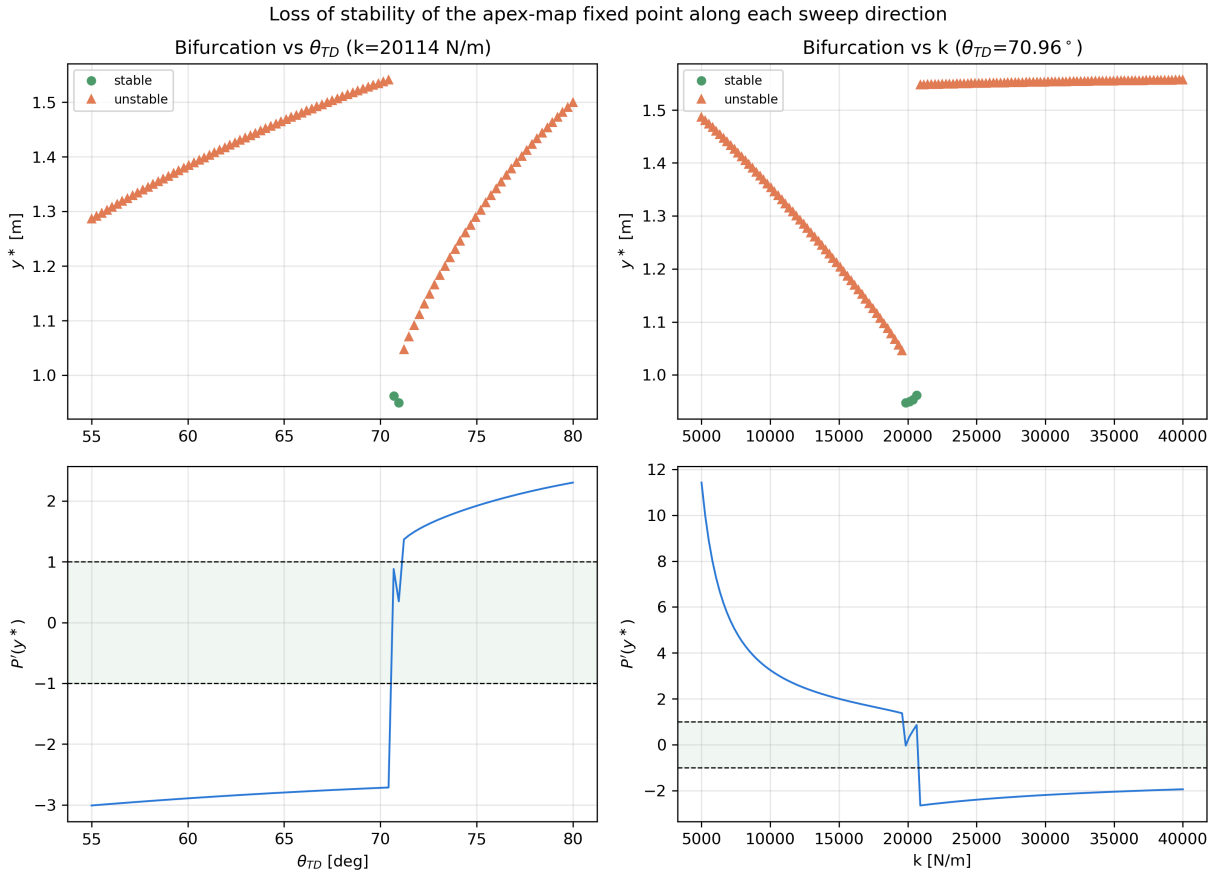

Figure 6: Bifurcation diagrams: fixed-point height y^* (top) and slope $P'(y^*)$ (bottom) vs. θ_{TD} at fixed k (left) and vs. k at fixed θ_{TD} (right), sliced through the representative stable cell of Section 4.4. Shaded band: $|P'(y^*)| < 1$ (stable). Marker shape doubles the stable/unstable color coding.

4.7 Stance-Phase Portrait

Section 1 motivates SLIP as building directly on the elastic pendulum studied in class. `stancePhasePortrait()` makes that connection concrete by plotting the stance-phase trajectory in the $(r, \dot{r})$ and $(\theta, \dot{\theta})$ phase planes for several touchdown angles at the same fixed apex energy (Figure 7). Each trajectory is an open arc, not a closed orbit: stance is a single finite-time pass through phase space between the touchdown

event (circle) and the liftoff event (square), not a free-running periodic oscillator – $r = r_0$ at both switching events, but $\dot{r}$ changes sign in between as the leg compresses and rebounds. Steeper touchdown angles produce shorter, less compressed arcs that stay close to $r = r_0$ throughout, consistent with a more vertical landing imparting less leg compression, while shallower angles compress the leg further and sweep through a wider range of θ.

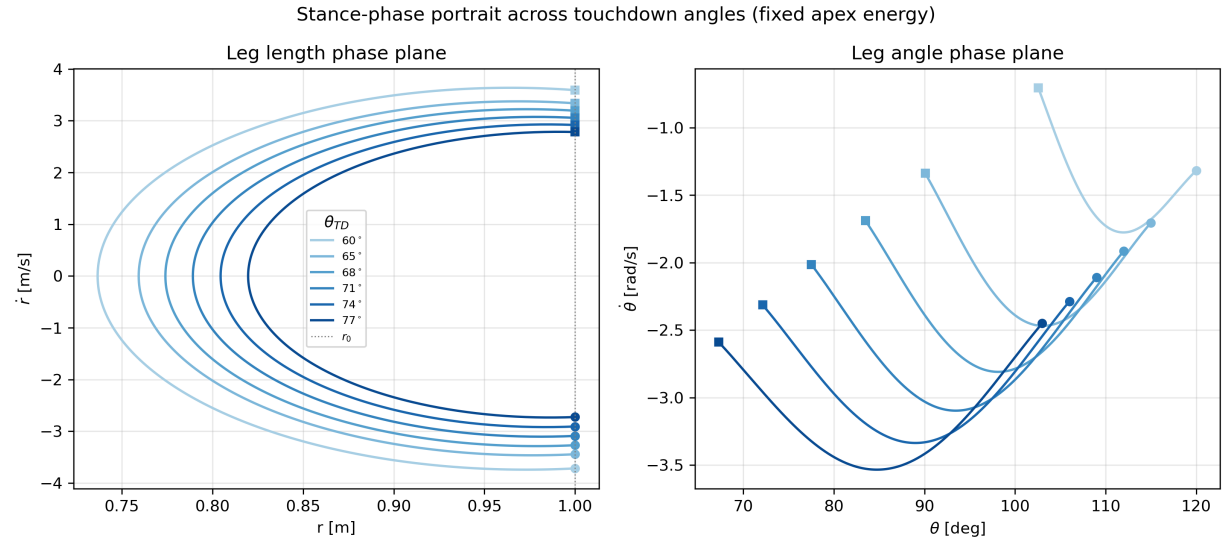

Figure 7: Stance-phase trajectories in the leg-length $(r, \dot{r})$ and leg-angle $(\theta, \dot{\theta})$ phase planes across touchdown angles $\theta_{TD} \in [60°, 77°]$ at fixed apex energy. Circle = touchdown, square = liftoffStateSwitch.

The sweep, its resolution check, and the iteration/bifurcation/phase-portrait figures above are all done; what remains is the interpretive write-up:

- Does the shape of the stable band (stiffness rising with touchdown angle, Figure 6) match the qualitative stiffness–speed trend reported by McMahon and Cheng [2]? Section 4.3's point (68°, 20000 N/m) is unstable and sits just outside the band at that stiffness – is that representative of the grid, or an outlier chosen before the sweep existed? What does it mean physically that the self-stable region is only a few cells wide even at full resolution – is steady hopping in this model fundamentally fragile, or is this an artifact of holding θ_{TD} fixed through flight (Section 3) with no swing-leg feedback?

5 Lessons Learned

This project proved to be incredibly informative. It allowed me to dive into a multi-stage dynamic system and taught me how to address the state transition conditions in a clear and defined way. It also expanded upon our study of geometric dynamics analysis by requiring a different mechanism for the stability comparison. Instead of having to analyze the whole system, we only have to look at the points in space that best define our motion. This also drove home that finding a periodic solution is not the same as finding a stable one: the fixed point of the apex return map exists whether or not it is attracting, so stability has to be checked separately – and for the validated parameters, it turned out the hopping gait was not stable at all.

It was also a great project for working through robotics-style problems. The inverted pendulum is a

very famous model used throughout all of robotics. But with the Lagrangian, this derivation can become relatively simple, opening the door for more accurate models. Working through the parameter sweep taught me a practical lesson about trusting numerical results, too: the first coarse grid over (k, θ_{TD}) turned up a stable band so thin it could have just been a grid-resolution artifact, and only after refining the grid and confirming the band held its width could I be confident that the narrow self-stable region was a real feature of the model rather than a numerical accident.

References

[1] R. Blickhan, "The spring-mass model for running and hopping," *Journal of Biomechanics*, vol. 22, no. 11–12, pp. 1217–1227, 1989.

[2] T. A. McMahon and G. C. Cheng, "The mechanics of running: how does stiffness couple with speed?," *Journal of Biomechanics*, vol. 23, suppl. 1, pp. 65–78, 1990.

[3] R. J. Full and D. E. Koditschek, "Templates and anchors: neuromechanical hypotheses of legged locomotion on land," *Journal of Experimental Biology*, vol. 202, no. 23, pp. 3325–3332, 1999.

[4] W. J. Schwind and D. E. Koditschek, "Approximating the stance map of a 2-DOF monoped runner," *Journal of Nonlinear Science*, vol. 10, pp. 533–568, 2000.

[5] U. Saranli, M. Buehler, and D. E. Koditschek, "RHex: a simple and highly mobile hexapod robot," *International Journal of Robotics Research*, vol. 20, no. 7, pp. 616–631, 2001.

[6] H. Geyer, A. Seyfarth, and R. Blickhan, "Compliant leg behaviour explains basic dynamics of walking and running," *Proceedings of the Royal Society B*, vol. 273, no. 1603, pp. 2861–2867, 2006.

Appendix: Code

`main.py` – touchdown map, stance-phase RHS, and the single-stance simulation and validation plots discussed in Section 4.1.

```python
import copy
import time
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares
from scipy.optimize import brentq

# Paramerts
m = 80
r_o = 1
k = 20000
g = 9.81
thetaTD = np.radians(68)
y_o = 1.2
v_o = 3.0

params = {'m' : m , 'k' : k , 'g' : g, 'r_o' : r_o , 'y_o' : y_o}


# ============================================================
# Stage 1 - Stance phase: touchdown -> standing -> liftoff
# First piece of the model: integrate a single stance phase and
# check that total mechanical energy is conserved through it.
# ============================================================

def touchdownState(x_o_dot , y_o, theta_TD, r_o, g):
    r = r_o
    theta = np.arctan2(r_o * np.sin(theta_TD) , -r_o * np.cos(theta_TD))

    x_TD_dot = x_o_dot
    y_TD_dot = - np.sqrt(2*g*(y_o - r_o * np.sin(theta_TD)))

    r_dot = x_TD_dot * np.cos(theta) + y_TD_dot * np.sin(theta)

    theta_dot = (-x_TD_dot * np.sin(theta) + y_TD_dot * np.cos(theta)) / r_o
```

```
37
38      return [r , theta, r_dot , theta_dot]
39
40  def standingState(t , s, params):
41      r_dot = s[2]
42      theta_dot = s[3]
43
44      r_ddot = s[0] * theta_dot**2 - params['g'] * np.sin(s[1]) - (params['k'] /
            params['m'])* (s[0] - params['r_o'])
45      theta_ddot = -(2/s[0]) * r_dot * theta_dot - (params['g'] / s[0]) * np.cos
            (s[1])
46
47      return [r_dot , theta_dot, r_ddot, theta_ddot]
48
49  def liftoffStateSwitch(t, s, params):
50      return params['k'] * (s[0] - params['r_o'])
51  liftoffStateSwitch.terminal = True
52  liftoffStateSwitch.direction = 1
53
54  def energy(s, params):
55      E = (1/2) * params['m'] * (s[2]**2 + s[0]**2 * s[3]**2) + params['m'] *
            params['g'] * s[0] * np.sin(s[1]) + (1/2) * params['k']* (s[0] -
            params['r_o'])**2
56
57      return E
58
59  # ---------- run a single stance phase for diagnostics ----------
60  # (mirrors the body of hop(), but keeps sol/s0 around for plotting)
61
62  def initialFunctionTest():
63      s0 = touchdownState(v_o, y_o, thetaTD, r_o, g)
64
65      t_span = (0.0, 1.0)
66      sol = solve_ivp(standingState, t_span, s0, args=(params,),
67                      events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
68                      dense_output=True, max_step=1e-3)
69
70      E = np.array([energy(sol.y[:, i], params) for i in range(sol.y.shape[1])])
71      drift = np.max(np.abs(E - E[0])) / np.abs(E[0])
72
73      # Plotting and data stuff, AI can fill in these gaps as simply learning to
            plot good is not hard and not my current goal
74
75      print("status:", sol.status)
76      print("liftoffStateSwitch time:", sol.t_events[0])
77      print("drift:", drift)
78
79      # ---------- diagnostics ----------
80      r_TD, th_TD, rdot_TD, thdot_TD = s0
81      s_LO = sol.y_events[0][0]
82      t_LO = sol.t_events[0][0]
83
84      print("--- touchdown state ---")
85      print(f"  r      = {r_TD:.6f} m")
86      print(f"  theta  = {np.degrees(th_TD):.4f} deg")
87      print(f"  r_dot  = {rdot_TD:.6f} m/s")
```

```
    print(f"  th_dot = {thdot_TD:.6f} rad/s")

    print("--- liftoffStateSwitch state ---")
    print(f"  status         = {sol.status}")
    print(f"  stance duration= {t_LO:.6f} s")
    print(f"  r              = {s_LO[0]:.6f} m")
    print(f"  theta          = {np.degrees(s_LO[1]):.4f} deg")
    print(f"  r_dot          = {s_LO[2]:.6f} m/s")
    print(f"  th_dot         = {s_LO[3]:.6f} rad/s")

    print("--- validation ---")
    print(f"  min leg length = {np.min(sol.y[0]):.6f} m")
    print(f"  max compression= {params['r_o'] - np.min(sol.y[0]):.6f} m")
    print(f"  E_0            = {E[0]:.6f} J")
    print(f"  relative drift = {drift:.3e}")

    # ---------- figures ----------
    t = sol.t
    r = sol.y[0]
    th = sol.y[1]
    x_com = r * np.cos(th)
    y_com = r * np.sin(th)

    fig, ax = plt.subplots(2, 2, figsize=(11, 8))

    ax[0, 0].plot(t, r, color='C3')
    ax[0, 0].axhline(params['r_o'], ls='--', lw=0.8, color='gray', label='
       $r_0$')
    ax[0, 0].set_xlabel('t [s]')
    ax[0, 0].set_ylabel('r [m]')
    ax[0, 0].set_title('Leg length')
    ax[0, 0].legend()

    ax[0, 1].plot(t, np.degrees(th), color='C0')
    ax[0, 1].axhline(90, ls='--', lw=0.8, color='gray', label='$90^\\circ$')
    ax[0, 1].set_xlabel('t [s]')
    ax[0, 1].set_ylabel(r'$\theta$ [deg]')
    ax[0, 1].set_title('Leg angle')
    ax[0, 1].legend()

    ax[1, 0].plot(x_com, y_com, color='C3', lw=2)
    ax[1, 0].plot(0, 0, 'kv', ms=10, label='foot (pinned)')
    ax[1, 0].plot(x_com[0], y_com[0], 'o', color='C0', label='touchdown')
    ax[1, 0].plot(x_com[-1], y_com[-1], 's', color='C2', label='
       liftoffStateSwitch')
    ax[1, 0].axhline(0, color='k', lw=1)
    ax[1, 0].set_xlabel('x [m]')
    ax[1, 0].set_ylabel('y [m]')
    ax[1, 0].set_title('CoM trajectory (foot frame)')
    ax[1, 0].axis('equal')
    ax[1, 0].legend()

    ax[1, 1].plot(t, E - E[0], color='C4')
    ax[1, 1].set_xlabel('t [s]')
    ax[1, 1].set_ylabel('$E - E_0$ [J]')
    ax[1, 1].set_title(f'Energy drift (rel. {drift:.2e})')
```

```
    ax[1, 1].ticklabel_format(axis='y', style='sci', scilimits=(0, 0))

    for a in ax.flat:
        a.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(os.path.join(SCRIPT_DIR, 'stance_validation.png'), dpi=200)
    plt.show()


# ============================================================
# Stage 2 - Apex state: ballistic flight after liftoff
# Convert the liftoff (r, theta) state back to Cartesian (x, y)
# and propagate the free-flight trajectory forward to the apex
# (the point where the vertical velocity goes to zero).
# ============================================================

def projectileApexState(s):
    r_o = s[0]
    theta_o = s[1]
    r_o_dot = s[2]
    theta_o_dot = s[3]

    # convert state from liftoff back to cartesian
    y_o = r_o * np.sin(theta_o)
    x_o = r_o * np.cos(theta_o)

    y_o_dot = r_o_dot * np.sin(theta_o) + theta_o_dot * r_o * np.cos(theta_o)
    x_o_dot = r_o_dot * np.cos(theta_o) - theta_o_dot * r_o * np.sin(theta_o)

    # Solve for when y velocity = 0, this is the highest point
    t_apex = y_o_dot / g

    # find corresponding x y x_dot y_dot values
    y_dot = -g*t_apex + y_o_dot
    x_dot = x_o_dot

    y = y_o - ((g/2) * t_apex**2) + y_o_dot * t_apex
    x = x_o + x_dot * t_apex

    return [x , y, x_dot , y_dot]


# ============================================================
# Stage 3 - Hop: touchdown -> stance -> liftoff -> apex
# Chains stages 1-2 together into a single map that takes an
# apex state and returns the next apex state, one hop later.
# ============================================================

def hop(apexState ,thetaTD, params):
    s0 = touchdownState(apexState[2],apexState[1], thetaTD, params['r_o'],
       params['g'])
    t_span = (0.0,1.0)

    sol = solve_ivp(standingState, t_span, s0, args=(params,),
                events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
                dense_output=True, max_step=1e-3)
```

```
197
198     s1 = projectileApexState(sol.y_events[0][0])
199
200     return s1
201
202 def energyHop(apexState, params):
203     E = (1/2) * params['m'] * (apexState[2]**2 + apexState[3]**2) + params['m'
        ] * params['g'] * apexState[1]
204     return E
205
206
207 # ============================================================
208 # Stage 4 - x is redundant: the apex-to-apex map is fully
209 # determined by total energy E and height y, not by x directly.
210 # ============================================================
211
212 def proveX_dotIsRedundant():
213     # E = (1/2)*m*(x_dot^2 + y_dot^2) + m*g*y
214     # ((E - mgy) * 2/m - y_dot**2))**.5 = x_dot
215     # Y_dot = 0
216     # ((E-mgy)*2/m)**.5 = x_dot
217     # (2E/m - 2gy)**.5 = x_dot
218
219     initialState = [0,y_o,v_o,0]
220     initialThetaTD = thetaTD
221
222     E1 = energyHop(initialState , params)
223
224
225     finalState = hop(initialState, initialThetaTD, params)
226     x_dot_pred = np.sqrt(2*E1/ params['m'] - 2*params['g']*finalState[1])
227     E2 = energyHop(finalState, params)
228
229     print(f"Energy Drift over 1 hop: {E2-E1}")
230     print(f"X_dot Drift over 1 hop: {finalState[2]-x_dot_pred}")
231
232 # ============================================================
233 # Stage 5 - Reduced 1-D apex map: fixed point and stability
234 # Stage 4 showed the hop only depends on E and y, so collapse the
235 # hop into a 1-D map y -> apexmap(y) and find the periodic apex
236 # height (fixed point), then check its local stability from the
237 # map's slope there.
238 # ============================================================
239
240 def apexmap(y , thetaTD, params, E):
241     y = np.asarray(y).item()
242
243     x_dot = np.sqrt(2*E/ params['m'] - 2*params['g']*y)
244
245     inputApexState = [0, y , x_dot , 0]
246     outputApexState = hop(inputApexState, thetaTD, params)
247
248     return outputApexState[1]
249
250 # ---------- find the fixed point of the apex map and its local stability
    ----------
```

```python

def findApexMapFixedPoint(params, thetaTD, y_o=y_o, v_o=v_o):
    initialState = [0,y_o,v_o,0]
    E = energyHop(initialState, params)
    f = lambda y: apexmap(y, thetaTD, params , E) - y

    # +.001 avoids singularities
    y_min = params['r_o'] * np.sin(thetaTD) + .001 # apex must be at/above touchdown height for this thetaTD
    # 2E/m - 2*g*y >= 0
    #  Y < 2E/ (2mg)
    y_max = 2*E / (2* params['m'] * params['g'])

    y_sample = np.linspace(y_min, y_max, 30)
    f_sample = np.array([f(yi) for yi in y_sample])

    sign_changes = np.where(np.diff(np.sign(f_sample)) != 0)[0]

    if sign_changes.size == 0:
        raise ValueError(f"no sign change in apex map for thetaTD={thetaTD}, k={params['k']}")

    y_min = y_sample[sign_changes[0]]
    y_max = y_sample[sign_changes[0] + 1]

    y_star: float = brentq(f, y_min, y_max)  # type: ignore[assignment]

    dy = max(1e-4 * abs(y_star), 1e-6)

    apexmapPrime = (apexmap(y_star + dy, thetaTD, params, E) - apexmap(y_star - dy, thetaTD, params, E)) / (2 * dy) # type: ignore

    # print(apexmapPrime)

    return (y_star, apexmapPrime)


# ============================================================
# Stage 6 - Grid search: fixed point and stability over (k, thetaTD)
# Each grid point needs its own params dict (deep-copied off the
# base params) so sweeping 'k' never mutates the shared dict that
# other stages/functions read from.
# ============================================================

def gridSearchFixedPoints(base_params, k_values, thetaTD_values, y_o=y_o, v_o=v_o):
    y_star_grid = np.full((len(k_values), len(thetaTD_values)), np.nan)
    slope_grid = np.full((len(k_values), len(thetaTD_values)), np.nan)
    n_failed = 0
    n_total = len(k_values) * len(thetaTD_values)

    # High-resolution sweeps can take well over an hour unattended, so print
    # a per-row progress/ETA line rather than going silent until the end.
    t_start = time.time()

    for i, k_val in enumerate(k_values):
```

```
        for j, theta_val in enumerate(thetaTD_values):
            trial_params = copy.deepcopy(base_params)
            trial_params['k'] = k_val
            try:
                y_star, slope = findApexMapFixedPoint(trial_params, theta_val,
                    y_o, v_o)
            except ValueError:
                n_failed +=1
                continue
            y_star_grid[i, j] = y_star
            slope_grid[i, j] = slope

        n_done = (i + 1) * len(thetaTD_values)
        elapsed = time.time() - t_start
        eta = elapsed * (n_total - n_done) / n_done
        print(f"  row {i+1}/{len(k_values)} (k={k_val:.0f}) done -- "
              f"{n_done}/{n_total} cells, {elapsed/60:.1f} min elapsed, "
              f"~{eta/60:.1f} min remaining")

    print(f"{n_failed}/{n_total} cells failed to find a fixed point")
    return y_star_grid, slope_grid



import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap, BoundaryNorm

def plotStabilitySweep(k_values, thetaTD_values, y_star_grid, slope_grid,
                       friction_floor_deg=59.0, save_path=None):
    """
    Classifies each grid cell as stable / unstable / no fixed point,
    overlays the friction-cone floor, and saves the deliverable figure.
    """
    K, THETA = np.meshgrid(k_values, thetaTD_values, indexing='ij')

    # 0 = no fixed point, 1 = unstable, 2 = stable
    classification = np.zeros_like(slope_grid)
    found = ~np.isnan(slope_grid)
    classification[found & (np.abs(slope_grid) < 1)] = 2
    classification[found & (np.abs(slope_grid) >= 1)] = 1

    cmap = ListedColormap(['#d9d9d9', '#e07b54', '#4c9a6a'])  # gray, orange,
        green
    bounds = [-0.5, 0.5, 1.5, 2.5]
    norm = BoundaryNorm(bounds, cmap.N)

    fig, ax = plt.subplots(figsize=(8, 6))
    mesh = ax.pcolormesh(THETA, K, classification, cmap=cmap, norm=norm,
        shading='auto')

    # friction-cone floor
    ax.axvline(friction_floor_deg, color='k', linestyle='--', linewidth=1.5)
    ax.axvspan(thetaTD_values.min(), friction_floor_deg, color='k', alpha
        =0.08)
    ax.text(friction_floor_deg + 0.5, k_values.max() * 0.97,
```

```
            r'$\theta_{TD} \geq %.0f^\circ$ (friction cone, $\mu$=0.6)' %
                friction_floor_deg,
            va='top', fontsize=9)

    ax.set_xlabel(r'Touchdown angle $\theta_{TD}$ (deg)')
    ax.set_ylabel(r'Leg stiffness $k$')
    ax.set_title('Self-stable running region')

    cbar = fig.colorbar(mesh, ax=ax, ticks=[0, 1, 2])
    cbar.ax.set_yticklabels(['No fixed point', 'Unstable', 'Stable'])

    fig.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=300)
    return fig, ax


# ============================================================
# Stage 7 - Local refinement:
# ============================================================

def localRefinementSweep(base_params, k_center, theta_center_deg,
                         k_half_width=2500.0, theta_half_width_deg=2.5,
                         n=20, y_o=y_o, v_o=v_o):
    """Rerun a small (k, thetaTD) window at finer resolution, zoomed on a
    stable cell found by the coarse sweep."""
    k_values = np.linspace(k_center - k_half_width, k_center + k_half_width, n
        )
    thetaTD_deg = np.linspace(theta_center_deg - theta_half_width_deg,
                              theta_center_deg + theta_half_width_deg, n)
    thetaTD_values = np.radians(thetaTD_deg)

    y_star_grid, slope_grid = gridSearchFixedPoints(base_params, k_values,
        thetaTD_values, y_o, v_o)
    return k_values, thetaTD_deg, y_star_grid, slope_grid


def stableBandWidth(slope_grid):
    """Longest run of contiguous stable (|slope| < 1) cells found along any
    single row (fixed k, varying thetaTD) or column (fixed thetaTD, varying
    k). A width of 1 on a fine grid means the band really is that thin at
    this resolution; >1 means the coarse sweep's one-cell staircase was an
    artifact of the coarse spacing."""
    stable = (~np.isnan(slope_grid)) & (np.abs(slope_grid) < 1)

    def longest_run(mask_1d):
        best = run = 0
        for is_stable in mask_1d:
            run = run + 1 if is_stable else 0
            best = max(best, run)
        return best

    row_runs = [longest_run(row) for row in stable]
    col_runs = [longest_run(col) for col in stable.T]
    return max(row_runs, default=0), max(col_runs, default=0)

```

```
408
409 # ============================================================
410 # Stage 8 - Deliverable figures: apex-map iteration (cobweb),
411 # bifurcation diagrams sliced from the (k, thetaTD) sweep, and
412 # the stance-phase portrait. These answer the write-up's own
413 # open questions about the sweep (Section 4.4's insert box):
414 # what does convergence/divergence of the apex map actually look
415 # like, and where exactly does the fixed point lose stability?
416 # ============================================================
417
418 def apexMapDomain(thetaTD, params, E):
419     """Same y-domain bounds used inside findApexMapFixedPoint(): apex height
420     must be at/above the touchdown height for this thetaTD, and low enough
421     that 2E/m - 2*g*y stays non-negative."""
422     y_min = params['r_o'] * np.sin(thetaTD) + 1e-3
423     y_max = E / (params['m'] * params['g'])
424     return y_min, y_max
425
426
427 def iterateApexMap(y0, thetaTD, params, E, n_iter=10, domain=None):
428     """Iterates y_(n+1) = apexmap(y_n) starting from y0, stopping early if an
429     iterate leaves the physically valid domain -- a real possibility right
430     next to an unstable fixed point, where the map blows up in a few hops."""
431     y_min, y_max = domain if domain is not None else (-np.inf, np.inf)
432     ys = [y0]
433     for _ in range(n_iter):
434         y_prev = ys[-1]
435         if not (y_min < y_prev < y_max):
436             break
437         try:
438             y_next = apexmap(y_prev, thetaTD, params, E)
439         except Exception:
440             break
441         if not np.isfinite(y_next):
442             break
443         ys.append(y_next)
444     return np.array(ys)
445
446
447 def _apexMapIterationPanel(ax_cobweb, ax_trace, thetaTD, params, E, y_star,
    slope,
448                            y0_offset, label, color, n_iter=10):
449     """One stable- or unstable-case pair: a cobweb diagram of P(y) with the
450     staircase iteration overlaid, plus the same iteration as y_n vs hop
451     index. Each panel is self-labeled (title + legend), so color here is
452     never the only channel identifying stable vs. unstable."""
453     y_min, y_max = apexMapDomain(thetaTD, params, E)
454     y_curve = np.linspace(y_min, y_max, 300)
455     P_curve = np.array([apexmap(y, thetaTD, params, E) for y in y_curve])
456
457     ax_cobweb.plot(y_curve, P_curve, color='#2a78d6', lw=1.5, label='$P(y)$')
458     ax_cobweb.plot(y_curve, y_curve, color='gray', ls='--', lw=1, label='$y_{n
        +1}=y_n$')
459     ax_cobweb.plot(y_star, y_star, marker='*', color='k', ms=12, zorder=5,
        label='$y^*$')
460
```

```
    ys = iterateApexMap(y_star + y0_offset, thetaTD, params, E, n_iter=n_iter,
        domain=(y_min, y_max))
    for n in range(len(ys) - 1):
        ax_cobweb.plot([ys[n], ys[n]], [ys[n], ys[n + 1]], color=color, lw=1,
            alpha=0.85)
        ax_cobweb.plot([ys[n], ys[n + 1]], [ys[n + 1], ys[n + 1]], color=color
            , lw=1, alpha=0.85)
    ax_cobweb.plot(ys[0], ys[0], 'o', color=color, ms=6, label='$y_0$')
    ax_cobweb.set_xlabel('$y_n$ [m]')
    ax_cobweb.set_ylabel('$y_{n+1}$ [m]')
    ax_cobweb.set_title(f"{label}\n$P'(y^*) = {slope:.2f}$")
    ax_cobweb.legend(fontsize=7, loc='best')
    ax_cobweb.grid(alpha=0.3)

    ax_trace.plot(range(len(ys)), ys, 'o-', color=color)
    ax_trace.axhline(y_star, color='k', ls='--', lw=0.8, label='$y^*$')
    ax_trace.set_xlabel('hop index $n$')
    ax_trace.set_ylabel('$y_n$ [m]')
    ax_trace.set_title('Apex height per hop')
    ax_trace.legend(fontsize=7)
    ax_trace.grid(alpha=0.3)
    return ys


def plotApexMapIteration(stable_case, unstable_case, save_path=None):
    """Side-by-side cobweb + iteration-trace figure contrasting the stable
    fixed point (Section 4.4/4.5's representative cell) with the unstable
    one (Section 4.3's validated parameter set) -- directly answers the
    write-up's request for an iteration plot showing convergence."""
    fig, axes = plt.subplots(2, 2, figsize=(11, 9))

    _apexMapIterationPanel(axes[0, 0], axes[1, 0], color='#4c9a6a', **
        stable_case)
    _apexMapIterationPanel(axes[0, 1], axes[1, 1], color='#e07b54', **
        unstable_case)

    fig.suptitle('Apex return map: cobweb iteration and per-hop apex height')
    fig.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=300)
    return fig, axes


def _bifurcationColumn(ax_top, ax_bot, x, y_star, slope, xlabel, title):
    found = ~np.isnan(slope)
    stable = found & (np.abs(slope) < 1)
    unstable = found & (np.abs(slope) >= 1)

    # Marker shape (o vs ^), not just color, distinguishes stable/unstable --
    # this green/orange pair reads fine for normal vision but is a poor
    # protanopia/deuteranopia pairing, so identity can't ride on hue alone.
    ax_top.plot(x[stable], y_star[stable], 'o', color='#4c9a6a', ms=5, label='
        stable')
    ax_top.plot(x[unstable], y_star[unstable], '^', color='#e07b54', ms=5,
        label='unstable')
    ax_top.set_ylabel(r'$y^\ast$ [m]')
```

```
    ax_top.set_title(title)
    ax_top.legend(fontsize=8)
    ax_top.grid(alpha=0.3)

    ax_bot.plot(x[found], slope[found], color='#2a78d6', lw=1.2)
    ax_bot.axhline(1, ls='--', color='k', lw=0.8)
    ax_bot.axhline(-1, ls='--', color='k', lw=0.8)
    ax_bot.axhspan(-1, 1, color='#4c9a6a', alpha=0.08)
    ax_bot.set_xlabel(xlabel)
    ax_bot.set_ylabel(r"$P'(y^\ast)$")
    ax_bot.grid(alpha=0.3)


def plotBifurcationDiagrams(k_values, thetaTD_deg_values, y_star_grid,
    slope_grid,
                                i_fixed_k, j_fixed_theta, save_path=None):
    """Two one-parameter bifurcation slices through the (k, thetaTD) sweep
    grid, both through the same representative stable cell used for the
    local refinement in Stage 7: y* (top) and P'(y*) (bottom) vs. thetaTD at
    fixed k (left column), and vs. k at fixed thetaTD (right column). Where
    the bottom row crosses +-1, the fixed point above loses stability -- a
    bifurcation of the apex map."""
    k_fixed_val = k_values[i_fixed_k]
    theta_fixed_val = thetaTD_deg_values[j_fixed_theta]

    fig, axes = plt.subplots(2, 2, figsize=(11, 8))

    _bifurcationColumn(axes[0, 0], axes[1, 0], thetaTD_deg_values,
                       y_star_grid[i_fixed_k, :], slope_grid[i_fixed_k, :],
                       xlabel=r'$\theta_{TD}$ [deg]',
                       title=f'Bifurcation vs $\\theta_{{TD}}$ (k={
                           k_fixed_val:.0f} N/m)')
    _bifurcationColumn(axes[0, 1], axes[1, 1], k_values,
                       y_star_grid[:, j_fixed_theta], slope_grid[:,
                           j_fixed_theta],
                       xlabel='k [N/m]',
                       title=f'Bifurcation vs k ($\\theta_{{TD}}$={
                           theta_fixed_val:.2f}$^\\circ$)')

    fig.suptitle('Loss of stability of the apex-map fixed point along each
        sweep direction')
    fig.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=300)
    return fig, axes


def stancePhasePortrait(thetaTD_deg_list, y_o, v_o, params, save_path=None):
    """Runs one stance phase per touchdown angle at fixed apex energy (same
    y_o, v_o for all) and plots the resulting trajectories in the (r, r_dot)
    and (theta, theta_dot) phase planes -- the SLIP-stance analogue of the
    elastic-pendulum phase portraits from lecture. Circle = touchdown,
    square = liftoffStateSwitch. thetaTD is a single-hue sequential
    parameter here (not a category), so it gets a light->dark blue ramp
    rather than a categorical palette."""
    fig, axes = plt.subplots(1, 2, figsize=(11, 5))
```

```
    cmap = plt.get_cmap('Blues')
    n = len(thetaTD_deg_list)

    for idx, theta_deg in enumerate(thetaTD_deg_list):
        theta_rad = np.radians(theta_deg)
        color = cmap(0.35 + 0.55 * idx / max(n - 1, 1))
        s0 = touchdownState(v_o, y_o, theta_rad, params['r_o'], params['g'])
        sol = solve_ivp(standingState, (0.0, 1.0), s0, args=(params,),
                        events=liftoffStateSwitch, rtol=1e-10, atol=1e-12,
                        dense_output=True, max_step=1e-3)
        if sol.status != 1 or len(sol.t_events[0]) == 0:
            continue
        r, th, rdot, thdot = sol.y
        axes[0].plot(r, rdot, color=color, lw=1.5, label=f'{theta_deg:.0f}$^\\
            circ$')
        axes[0].plot(r[0], rdot[0], 'o', color=color, ms=5)
        axes[0].plot(r[-1], rdot[-1], 's', color=color, ms=5)

        axes[1].plot(np.degrees(th), thdot, color=color, lw=1.5)
        axes[1].plot(np.degrees(th[0]), thdot[0], 'o', color=color, ms=5)
        axes[1].plot(np.degrees(th[-1]), thdot[-1], 's', color=color, ms=5)

    axes[0].axvline(params['r_o'], ls=':', color='gray', lw=0.8, label='$r_0$'
        )
    axes[0].set_xlabel('r [m]')
    axes[0].set_ylabel(r'$\dot r$ [m/s]')
    axes[0].set_title('Leg length phase plane')
    axes[0].legend(title=r'$\theta_{TD}$', fontsize=7)
    axes[0].grid(alpha=0.3)

    axes[1].set_xlabel(r'$\theta$ [deg]')
    axes[1].set_ylabel(r'$\dot\theta$ [rad/s]')
    axes[1].set_title('Leg angle phase plane')
    axes[1].grid(alpha=0.3)

    fig.suptitle('Stance-phase portrait across touchdown angles (fixed apex
        energy)')
    fig.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=300)
    return fig, axes


# ============================================================
# Run
# ============================================================

import os
import warnings

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

# All Stage 8 deliverable figures are saved directly into the latex source
# tree so there is no manual copy step before compiling.
GRAPH_DIR = os.path.normpath(os.path.join(SCRIPT_DIR, '..', 'latex', 'Figures'
    , 'Graphs'))
```

```
os.makedirs(GRAPH_DIR, exist_ok=True)

RESULTS_PATH = os.path.join(SCRIPT_DIR, "sweep_results.npz")
RERUN_SWEEP = False  # set True to force a fresh sweep even if a cached file exists


def runOrLoadSweep(base_params, k_values, thetaTD_values, y_o, v_o,
                   results_path=RESULTS_PATH, rerun=RERUN_SWEEP):
    if not rerun and os.path.exists(results_path):
        data = np.load(results_path)
        cached_k = data['k_values']
        cached_theta = data['thetaTD_values']
        if (cached_k.shape == k_values.shape and np.allclose(cached_k, k_values)
                and cached_theta.shape == thetaTD_values.shape
                and np.allclose(cached_theta, thetaTD_values)):
            print(f"Loaded cached sweep from {results_path}")
            return data['y_star_grid'], data['slope_grid']
        else:
            print("Cached grid does not match requested k_values/thetaTD_values - rerunning.")

    print("Running fresh sweep...")
    y_star_grid, slope_grid = gridSearchFixedPoints(base_params, k_values, thetaTD_values, y_o, v_o)
    np.savez(results_path,
             k_values=k_values,
             thetaTD_values=thetaTD_values,
             y_star_grid=y_star_grid,
             slope_grid=slope_grid)
    print(f"Saved sweep to {results_path}")
    return y_star_grid, slope_grid

if __name__ == "__main__":
    warnings.filterwarnings('error', category=np.exceptions.ComplexWarning)

    # Bumped from 40x40: the local refinement check below showed the coarse
    # grid's one-cell-wide stable "staircase" was a resolution artifact, not
    # a real measure-zero band -- it thickened into a connected patch at 4x
    # finer spacing. This resolution matches that refinement window's cell
    # spacing (~0.26 deg, ~265 stiffness) across the full sweep range.
    # ~12,600 cells; at ~0.47 s/cell (measured on this machine) that's
    # roughly 90-100 minutes. Grab coffee.
    k_sweep = np.linspace(5000, 40000, 133)
    thetaTD_sweep_deg = np.linspace(55, 80, 95)
    thetaTD_sweep_rad = np.radians(thetaTD_sweep_deg)

    y_star_grid, slope_grid = runOrLoadSweep(params, k_sweep, thetaTD_sweep_rad, y_o, v_o)
    print("y_star grid:\n", y_star_grid)
    print("slope grid:\n", slope_grid)

    fig, ax = plotStabilitySweep(k_sweep, thetaTD_sweep_deg, y_star_grid, slope_grid,
                                 friction_floor_deg=59.0,
```

```
                                    save_path=os.path.join(SCRIPT_DIR, '
                                        stability_sweep.png'))

    coarse_row_run, coarse_col_run = stableBandWidth(slope_grid)
    print(f"coarse grid stable band width: {coarse_row_run} cell(s) along
        theta, "
          f"{coarse_col_run} cell(s) along k")

    # ---- local refinement: zoom a 20x20 window (5 deg x 5000 stiffness,
    # matching one dip below the friction-cone floor at 59 deg so we're not
    # zooming into a physically-excluded corner) onto a stable cell from the
    # coarse sweep and see if the band thickens at 4x the resolution ----
    stable_coarse = (~np.isnan(slope_grid)) & (np.abs(slope_grid) < 1)
    stable_idx = np.argwhere(stable_coarse)
    if stable_idx.size == 0:
        print("no stable cells found in the coarse sweep -- skipping
            refinement")
    else:
        i, j = stable_idx[len(stable_idx) // 2]  # a representative stable
            cell, not an edge case
        k_center = k_sweep[i]
        theta_center_deg = thetaTD_sweep_deg[j]
        print(f"refining around k={k_center:.1f}, thetaTD={theta_center_deg:.2
            f} deg")

        k_fine, theta_fine_deg, y_star_fine, slope_fine = localRefinementSweep
            (
            params, k_center, theta_center_deg,
            k_half_width=2500.0, theta_half_width_deg=2.5, n=20, y_o=y_o, v_o=
                v_o)

        fine_row_run, fine_col_run = stableBandWidth(slope_fine)
        print(f"refined grid stable band width: {fine_row_run} cell(s) along
            theta, "
              f"{fine_col_run} cell(s) along k (out of {len(theta_fine_deg)}x{
                  len(k_fine)} cells, "
              f"~{(theta_fine_deg[1]-theta_fine_deg[0]):.3f} deg / {(k_fine
                  [1]-k_fine[0]):.1f} stiffness spacing)")

        if fine_row_run <= 1 and fine_col_run <= 1:
            print("band stays one-cell-wide at 4x finer resolution -- looks
                like a real, "
                  "genuinely narrow stability margin, not a resolution
                      artifact.")
        else:
            print("band thickens at finer resolution -- the coarse sweep's
                staircase was "
                  "a grid artifact; bump the full sweep's resolution before
                      finalizing.")

        plotStabilitySweep(k_fine, theta_fine_deg, y_star_fine, slope_fine,
                           friction_floor_deg=59.0,
                           save_path=os.path.join(SCRIPT_DIR, '
                               stability_refinement.png'))

        # ---- Stage 8: deliverable figures for the write-up ----
```

```
        trial_stable_params = copy.deepcopy(params)
        trial_stable_params['k'] = k_center
        y_star_stable = y_star_grid[i, j]
        slope_stable = slope_grid[i, j]

        y_star_unstable, slope_unstable = findApexMapFixedPoint(params,
            thetaTD)
        E_fixed = energyHop([0, y_o, v_o, 0], params)

        stable_case = dict(
            thetaTD=np.radians(theta_center_deg), params=trial_stable_params,
                E=E_fixed,
            y_star=y_star_stable, slope=slope_stable, y0_offset=0.05,
            label=f'Stable case ($k\\approx{k_center:.0f}$ N/m, '
                  f'$\\theta_{{TD}}\\approx{theta_center_deg:.2f}^\\circ$)')
        unstable_case = dict(
            thetaTD=thetaTD, params=params, E=E_fixed,
            y_star=y_star_unstable, slope=slope_unstable, y0_offset=0.02,
            label=f'Unstable case ($k={params["k"]:.0f}$ N/m, '
                  f'$\\theta_{{TD}}={np.degrees(thetaTD):.0f}^\\circ$)')

        plotApexMapIteration(stable_case, unstable_case,
                             save_path=os.path.join(GRAPH_DIR, '
                                apex_map_iteration.png'))

        plotBifurcationDiagrams(k_sweep, thetaTD_sweep_deg, y_star_grid,
            slope_grid, i, j,
                                save_path=os.path.join(GRAPH_DIR, '
                                   bifurcation_diagrams.png'))

        stancePhasePortrait([60, 65, 68, 71, 74, 77], y_o, v_o, params,
                            save_path=os.path.join(GRAPH_DIR, '
                               stance_phase_portrait.png'))

        print(f"Stage 8 figures saved to {GRAPH_DIR}")
```